COURSE PRACTICE / 2026.11

微处理器

参考答案与解析

104 道分章填空题 · 23 道汇编程序填空题 · 共 303 空

依据：2026 版课本、课程课件、往届带答案原卷、课堂记录与三次实验。重点参照 2026 年 10 月 19 日两份 Lec12 复习记录。

- 只编入普通填空和汇编程序填空，不含简答题。第二章作为读硬件的基础，编程不要求编写Verilog。
- 主线依据2026-10-19两份重点记录；复习提点不等于确定考点。本套为学习练习，不能预测试卷。
- 教材页码为正文印刷页，PDF页序=正文页+5。课件页码直接按PDF页序。每题保留来源。
- 题目用ARMASM课堂风格，面向Cortex-M0；程序变量、地址、电平和协议均以各题题面为准。
- 自动核对只识别参考答案及已列同义写法，不证明程序语义；等价写法可对照解析后自评。
- 已校订课件和现有试卷整理中的明显问题：软核“实现工艺”、F(50)溢出、#R0笔误、C调用寄存器保护、栈对齐、字符串容量与M0访存写法。
- 专项组卷只含15个普通填空+5/10/10个程序空，共80分，不包含原考试20分简答，不能作为完整模拟卷。

练习顺序

先按教材章节查漏，再练 P01-P05 纯汇编、P06-P11 混合编程、P12-P18 软硬件综合，以及新增的 P19-P23 C 调用汇编迁移题。C调用示例已经给出，只填汇编。答案编号与网页一致。

打印建议：练习册与答案册分开装订。程序空按题面指令格式填写；答案册提供逐空参考、边界案例和语义解析。

目录与专项组卷

第1章 片上系统设计方法概述 10题 / p3

第2章 硬件描述语言与数字系统基础 10题 / p5

第3章 微处理器原理与 ARM 内核 22题 / p7

第4章 Thumb 指令与 ARM 汇编程序设计 22题 / p10

第5章 Cortex-M0 片上系统与总线 20题 / p13

第6章 IP 集成与软硬件协同 20题 / p16

程序填空

纯汇编 P01-P05 / p19

C 与汇编混合编程 P06-P11 / p24

软硬件综合与接口驱动 P12-P18 / p30

C调用汇编迁移 P19-P23 / p37

4 套专项练习 · 每套 80 分

每套由 15 个普通填空和 3 道程序题组成，程序空数为 5/10/10。每空 2 分。仅为本题库的组合，不包含简答题，也不代表完整考试。

专项 A · 基础与 GPIO

填空：F1-04、F1-09、F3-03、F3-06、F3-08、F3-13、F3-15、F3-16、F4-15、F4-21、F5-08、F5-11、F5-14、F6-02、F6-08

编程：P01、P08、P13

专项 B · 调用与显示

填空：F1-03、F2-03、F3-07、F3-09、F3-14、F3-17、F3-20、F4-13、F4-16、F4-20、F5-07、F5-15、F5-18、F6-10、F6-13

编程：P03、P09、P15

专项 C · 综合与加速器

填空：F1-06、F2-10、F3-05、F3-12、F3-18、F4-08、F4-11、F4-22、F5-09、F5-10、F5-17、F5-20、F6-15、F6-16、F6-18

编程：P02、P10、P16

专项 D · C调用汇编迁移

填空：F3-08、F3-09、F3-10、F3-17、F3-18、F4-03、F4-04、F4-09、F4-10、F4-11、F4-12、F4-15、F4-20、F4-21、F4-22

编程：P19、P22、P20

资料回查

普通填空题后的 p 为教材正文页，PDF 页序需加 5。课件参考写明 PDF 页序。实验路径和课堂时间戳可在网页中展开查看。

CHAPTER 01 / 分章填空

片上系统设计方法概述

F1-01 教材巩固

SoC 的中文名称是片上系统。

解析：SoC 将处理器、存储器、接口及专用模块等集成为系统。

来源：教材第1章 · 正文 p7-8

F1-02 教材巩固

购买处理器 IP 得到的是可集成的设计数据，还须与存储器及外设等组合才能形成实际 SoC。

解析：内核 IP 不是已经包含所有外围功能的完整 SoC 芯片。

来源：教材第1章 · 正文 p12

F1-03 教材巩固

IP 核按交付形式分为软核、固核和硬核。

解析：固核在软核与硬核之间作折衷。

来源：教材第1章 · 正文 p12

F1-04 复习课件例题

由于软核不依赖于最终的实现工艺，因此具有较大灵活性。

解析：按复习课件原句填“工艺”；不要照搬 example.md 的“硬件实现”。

来源：教材第1章 · 正文 p12；期末复习总结 · 填空例题与答案

F1-05 教材巩固

软核通常以HDL源文件的形式交付。

解析：描述功能而未固定最终物理实现。

来源：教材第1章 · 正文 p12

F1-06 教材巩固

将功能分配给软件或硬件实现，是软硬件协同设计中的功能划分。

解析：划分时需要综合考虑性能、成本、功耗等约束。

来源：教材第1章 · 正文 p9-10

F1-07 教材巩固

EDA 的中文含义是电子设计自动化。

解析：EDA 工具支持设计输入、综合与实现等环节。

来源：教材第1章 · 正文 p10-11

F1-08 教材巩固

从系统总体需求逐步分解到各模块的设计方法称为自顶向下设计。

解析：先划分系统，再细化各模块及其接口。

来源：教材第1章 · 正文 p10-11

F1-09 教材巩固

多个处理器各有 Cache 而共享数据时，需要解决 Cache 的一致性问题。

解析：一个副本被修改后，其他副本可能过期。

来源：教材第1章 · 正文 p13-15

F1-10 教材巩固

多处理器共享总线时，同时访存会产生总线竞争，限制扩展性。

解析：复习总结要求把总线竞争与缓存一致性联系起来理解。

来源：教材第1章 · 正文 p13-15

CHAPTER 02 / 分章填空

硬件描述语言与数字系统基础

F2-01 教材巩固

Verilog 模块用 module 开始，用endmodule结束。

解析：模块对应有明确端口和内部功能关系的硬件实体。

来源：教材第2章 · 正文 p26-27

F2-02 教材巩固

组合逻辑中的连续赋值常使用关键字assign。

解析：连续赋值会随右侧信号变化而更新。

来源：教材第2章 · 正文 p29-30

F2-03 教材巩固

在时序 always 块中，为表达寄存器同时更新，通常使用非阻塞赋值。

解析：非阻塞赋值有助于正确表达触发器的同步更新。

来源：教材第2章 · 正文 p30-31

F2-04 教材巩固

仅在 posedge clk 时更新的存储单元，其状态通常在时钟上升沿改变。

解析：描述上升沿触发的时序逻辑。

来源：教材第2章 · 正文 p46-47

F2-05 教材巩固

always @(posedge clk or negedge rst_n) 中 rst_n 是低有效的异步复位。

解析：复位下降沿无需等待时钟上升沿即可触发。

来源：教材第2章 · 正文 p46-47

F2-06 教材巩固

Moore 状态机的输出仅与当前状态有关。

解析：Mealy 输出还与当前输入有关。

来源：教材第2章 · 正文 p85-86

F2-07 教材巩固

Mealy 状态机的输出与当前状态以及当前输入有关。

解析：注意与 Moore 的区别。

来源：教材第2章 · 正文 p85-86

F2-08 教材巩固

数码管显示中，选择哪个数字图形的是段选信号；选择哪一位亮的是位选信号。

解析：软件需按实际电平有效性分别写段选和位选端口。

来源：教材第2章 · 正文 p65-70

F2-09 教材巩固

PWM 调光主要通过改变高电平持续时间与周期之比，即占空比，来改变平均亮度。

解析：固定周期内调整导通时间。

来源：教材第2章 · 正文 p87-91

F2-10 教材巩固

加速器硬件常划分为执行运算的数据通路与决定各步动作的控制逻辑。

解析：第二章以 GCD 为例；期末编程侧重读懂电路后写汇编驱动。

来源：教材第2章 · 正文 p109-116

CHAPTER 03 / 分章填空

微处理器原理与 ARM 内核

F3-01 教材巩固

冯·诺依曼计算机的五大部件为运算器、控制器、存储器、输入设备和输出设备。

解析：处理器核心通常由运算器和控制器组成。

来源：教材第3章 · 正文 p122-125

F3-02 教材巩固

计算机内部三总线是地址总线、数据总线和控制总线。

解析：分别传地址、数据及读写控制等信息。

来源：教材第3章 · 正文 p124-125

F3-03 教材巩固

指令的基本执行过程是取指、译码和执行。

解析：处理器通过解释指令序列在时间上逐步执行操作。

来源：教材第3章 · 正文 p128-129

F3-04 教材巩固

对程序员可见的机器属性，如指令系统，属于计算机的体系结构。

解析：组织关注如何连接与配合，实现关注物理技术。

来源：教材第3章 · 正文 p121-123

F3-05 教材巩固

指令系统是软件与硬件之间的约定。

解析：软件按指令规范编程，硬件保证实现对应操作。

来源：教材第3章 · 正文 p130-132

F3-06 教材巩固

机器指令中，指明完成何种操作的字段是操作码。

解析：不要与指明数据及其位置的操作数字段混淆。

来源：教材第3章 · 正文 p130-132

F3-07 真题改编

一条指令的地址字段和相关寻址方式会影响处理器的组织结构。

解析：对应往届填空原意。

来源：教材第3章 · 正文 p130-132；往届 A 卷 · 填空题

F3-08 教材巩固

操作数直接包含在指令编码中的寻址方式是立即寻址。

解析：如 MOVS R0,#7，7 在指令中直接给出。

来源：教材第3章 · 正文 p134-135

F3-09 教材巩固

操作数在指定通用寄存器内的方式称为寄存器寻址。

解析：如 MOV R0,R1 从 R1 取值。

来源：教材第3章 · 正文 p135

F3-10 教材巩固

寄存器内存放操作数的存储地址，按该地址访存，称为寄存器间接寻址。

解析：如 LDR R0,[R1]；R1 是地址，R0 接收内容。

来源：教材第3章 · 正文 p135-136

F3-11 教材巩固

以 PC 为基准加偏移形成转移目标的方式称为相对寻址。

解析：标号在汇编时可被转换为相对偏移。

来源：教材第3章 · 正文 p138

F3-12 教材巩固

RISC 的访存通常采用Load-Store体系，仅加载与存储指令访问存储器。

解析：运算指令通常在寄存器间完成。

来源：教材第3章 · 正文 p139-140

F3-13 教材巩固

Cortex-M0 中 R13、R14、R15 分别称为 SP、LR、PC。

解析：SP 为栈指针，LR 保存函数返回信息，PC 为程序计数器。

来源：教材第3章 · 正文 p148-149

F3-14 教材巩固

执行 BL 或 BLX 调用时，函数返回地址被写入LR。

解析：非叶子函数须在进一步调用前保存自己的 LR。

来源：教材第3章 · 正文 p149

F3-15 真题改编

Cortex-M0 中 EPSR 的 T 位表示Thumb状态。

解析：Cortex-M0 不支持切换到 ARM 状态。

来源：教材第3章 · 正文 p150；往届 A 卷 · 填空题

F3-16 真题改编

EPSR 的 T 位被置零会在后续执行时引发HardFault异常。

解析：题意以课程讲义描述为准。

来源：教材第3章 · 正文 p150；往届 A 卷 · 填空题

F3-17 教材巩固

APSR 的 N、Z、C、V 中，表示结果为零的标志是Z。

解析：CMP 会更新条件标志，BEQ 测试 Z=1。

来源：教材第3章 · 正文 p150

F3-18 教材巩固

APSR 中表示有符号运算溢出的标志为V。

解析：C 是无符号进位或减法无借位语义，不等于有符号溢出。

来源：教材第3章 · 正文 p150

F3-19 教材巩固

按本课程 Cortex-M0 的字节序，0x12345678 存到地址 A 后，最低地址字节为 0x78。

解析：小端序把数值最低有效字节放在最低地址。

来源：教材第3章 · 正文 p153

F3-20 教材巩固

Cortex-M0 栈采用满递减模型，PUSH 后栈向低地址增长。

解析：SP 指向最后压入的数据。

来源：教材第3章 · 正文 p137-138

F3-21 教材巩固

Cortex-M0 的硬件异常处理使用MSP栈指针。

解析：课程裸机实验主要使用 MSP。

来源：教材第3章 · 正文 p149

F3-22 复习课件例题

程序倾向于再次访问最近访问的数据或指令，复习课件例题将其称为引用局部性。

解析：按课件完整表述是“引用局部性”；从具体行为看属于时间局部性。

来源：教材第3章 · 正文 p125-126；期末复习总结 · 填空例题与答案

CHAPTER 04 / 分章填空

Thumb 指令与 ARM 汇编程序设计

F4-01 教材巩固

把存储器中的一个字加载到寄存器，常使用LDR指令。

解析：LDR 为加载，STR 为存储。

来源：教材第4章 · 正文 p175-177

F4-02 教材巩固

将寄存器的一个字写入存储器，常使用STR指令。

解析：方向是寄存器到存储器或映射端口。

来源：教材第4章 · 正文 p175-177

F4-03 教材巩固

读取一个无符号字节并零扩展，常使用LDRB指令。

解析：不要用一次字读取替代字符读取。

来源：教材第4章 · 正文 p175-177

F4-04 教材巩固

LDR R0,[R1,#4] 的有效地址为 R1 的值加4字节。

解析：题设为按字节编址，偏移单位是字节。

来源：教材第4章 · 正文 p175-177

F4-05 教材巩固

LSRS 右移时，高位补0。

解析：适合无符号位操作，区别于 ASRS 符号扩展。

来源：教材第4章 · 正文 p168-170

F4-06 教材巩固

ASRS 右移时，高位补原数据的符号位。

解析：算术右移用于有符号数据。

来源：教材第4章 · 正文 p168-170

F4-07 教材巩固

实现按位与运算并更新标志的指令是ANDS。

解析：例如 ANDS R2,R2,R3。

来源：教材第4章 · 正文 p166-168

F4-08 教材巩固

只做按位与测试、更新条件标志而不保存结果，可使用TST。

解析：Cortex-M0 练习使用寄存器掩码形式 TST Rn,Rm。

来源：教材第4章 · 正文 p171-172

F4-09 教材巩固

CMP R0,#0 后，R0 为零时可使用BEQ跳转。

解析：CMP 更新 Z，BEQ 在 Z=1 时转移。

来源：教材第4章 · 正文 p165-166

F4-10 教材巩固

CMP R0,#0 后，R0 非零时可使用BNE跳转。

解析：与 BEQ 相对。

来源：教材第4章 · 正文 p165-166

F4-11 教材巩固

比较两个无符号数，R0 小于 R1 时可用 BCC，也称BLO。

解析：有符号小于使用 BLT；需按数据类型选择。

来源：教材第4章 · 正文 p172-174

F4-12 教材巩固

从不再调用其他函数的叶子函数返回，可执行 BX LR。

解析：BX LR 将控制权交回调用方。

来源：教材第4章 · 正文 p172-174

F4-13 教材巩固

在函数中执行另一次 BL 之前，通常要保存当前LR，否则会丢失返回路径。

解析：嵌套调用会覆盖 LR。

来源：教材第4章 · 正文 p173-174

F4-14 教材巩固

汇编源程序中定义代码段或数据段，ARMASM 使用AREA伪指令。

解析：是汇编期的组织信息，不是 CPU 运行期操作。

来源：教材第4章 · 正文 p193-194

F4-15 教材巩固

向其他源文件公开当前文件定义的函数符号，使用EXPORT伪指令。

解析：IMPORT 用于导入外部符号，方向不同。

来源：教材第4章 · 正文 p194-195

F4-16 教材巩固

汇编代码调用外部 C 函数前，可用IMPORT声明外部符号。

解析：区分导入与导出。

来源：教材第4章 · 正文 p194-195

F4-17 教材巩固

ARMASM 定义一个 32 位常数数据项使用DCD。

解析：如向量表的初始 SP 和各处理函数地址。

来源：教材第4章 · 正文 p189

F4-18 教材巩固

ARMASM 为栈等区域保留字节空间，可使用SPACE。

解析：常见形式 Stack_Mem SPACE Stack_Size。

来源：教材第4章 · 正文 p189

F4-19 教材巩固

LDR R0,=0x40000000 中带等号的形式属于加载常数的伪指令。

解析：汇编器可能用文字池或其他合适指令实现。

来源：教材第4章 · 正文 p181-182

F4-20 教材巩固

按课程常用调用规则，前四个整型参数依次放在 R0、R1、R2、R3。

解析：调用者不能假定 R0-R3 在调用后保持原值。

来源：教材第4章 · 正文 p207-209

F4-21 教材巩固

普通 32 位整型函数返回值通常放在R0寄存器。

解析：64 位返回有不同规则；本题限定普通 32 位值。

来源：教材第4章 · 正文 p207-209

F4-22 教材巩固

函数借用 R4-R7 等被调用者保存寄存器后，返回前必须将它们恢复。

解析：用 PUSH/POP 等方式满足保存约定；跨 C 调用还须注意栈对齐。

来源：教材第4章 · 正文 p207-209

CHAPTER 05 / 分章填空

Cortex-M0 片上系统与总线

F5-01 真题改编

在典型主从式总线系统中，处理器通常作为主机方主动发起传输。

解析：从机可请求服务，但不能以从机身份主动发起总线事务。

来源：教材第5章 · 正文 p217-220；往届 A 卷 · 填空题

F5-02 教材巩固

地址总线在本课程单主机系统中，由主机发送地址到从机。

解析：读数据从从机流向主机，写数据反向。

来源：教材第5章 · 正文 p217-220

F5-03 教材巩固

同步总线使用共同的时钟信号协调传输。

解析：异步总线通过握手协调不同速度的设备。

来源：教材第5章 · 正文 p221-224

F5-04 教材巩固

异步总线主要通过请求与应答等握手信号协调传输。

解析：总线异步与串行异步的起止位概念应区分。

来源：教材第5章 · 正文 p223-224

F5-05 教材巩固

半同步总线可通过插入等待状态适应慢设备。

解析：总线有时钟，并允许传输延长。

来源：教材第5章 · 正文 p224

F5-06 复习课件例题

AHB-Lite 传输分为地址相位与数据相位。

解析：单笔事务至少跨两个相位；流水重叠时不等于每笔吞吐都要两个周期。

来源：教材第5章 · 正文 p226-227；期末复习总结 · 填空例题与答案

F5-07 教材巩固

AHB-Lite 的 HWRITE=1 表示写操作。

解析：HWRITE=0 表示读。

来源：教材第5章 · 正文 p225-226

F5-08 真题改编

AHB-Lite 的 HREADY=0 表示传输进入等待状态。

解析：应保持当前传输直到可完成；不能把等待当作完成。

来源：教材第5章 · 正文 p226-227；往届 A 卷 · 填空题

F5-09 教材巩固

HTRANS 的 NONSEQ 编码为二进制10。

解析：HTRANS[1]=1 表示 NONSEQ 或 SEQ 有效传输；IDLE/BUSY 不应当成有效访问。

来源：教材第5章 · 正文 p225-226

F5-10 教材巩固

在 AHB-Lite 中，32 位字传输对应 HSIZE 的十进制值2。

解析：HSIZE 是每次传输字节数的 log2，2 对应 4 字节。

来源：教材第5章 · 正文 p225-226

F5-11 真题改编

地址空间是地址编码的集合。

解析：地址空间不是“已安装的物理 RAM 容量”。

来源：教材第5章 · 正文 p228-229；往届 A 卷 · 填空题

F5-12 教材巩固

32 位字节地址能够表示的地址空间大小为4GiB。

解析：2^32 字节等于 4 GiB；并非一定有 4 GiB 实际存储器。

来源：教材第5章 · 正文 p229

F5-13 教材巩固

存储器与 I/O 使用同一套地址编码，称为统一编址。

解析：通过普通访存指令操作映射 I/O。

来源：教材第5章 · 正文 p228-229

F5-14 真题改编

统一编址使 I/O 访问可以使用较丰富的寻址方式。

解析：不必为每类 I/O 另设计专用指令。

来源：教材第5章 · 正文 p229；往届 A 卷 · 填空题

F5-15 教材巩固

在本课程未重定位的向量表中，地址 0x00000000 保存初始SP值。

解析：0x00000004 保存复位处理函数入口。

来源：教材第5章 · 正文 p230,239-240

F5-16 复习课件例题

向量表基址为 0 时，HardFault 异常向量项位于 0x0000000C。

解析：是向量项存放地址，不是固定的 HardFault 处理函数入口。

来源：教材第5章 · 正文 p230；期末复习总结 · 填空例题与答案

F5-17 教材巩固

地址译码电路将地址编码转换成相应模块的片选信号。

解析：低地址位还能选择模块内的具体寄存器。

来源：教材第5章 · 正文 p231-234

F5-18 实验二

若 HADDR[31:4]==28'h4000001，则该外设的起始字节地址是 0x40000010。

解析：低 4 位未参与片选比较，该窗口覆盖 0x40000010-0x4000001F。

来源：教材第5章 · 正文 p231-234；实验二要求；实验二寄存器与电平定义

F5-19 实验二

上述 16 字节窗口内，偏移 0x04 的 32 位寄存器地址为 0x40000014。

解析：基地址 0x40000010 加字节偏移 4。

来源：教材第5章 · 正文 p231-234；实验二要求；实验二寄存器与电平定义

F5-20 实验三

AHB 写操作中，应先锁存地址相位信息，再在对应的数据相位用 HWDATA 写目标寄存器。

解析：不能拿下一笔地址配当前数据；实验三接口用 addr 与 wr_pending 配对。

来源：教材第5章 · 正文 p226-227；实验三要求；实验三加速器寄存器接口

CHAPTER 06 / 分章填空

IP 集成与软硬件协同

F6-01 教材巩固

软件直接访问的接口寄存器，按作用通常分为数据端口、状态端口和控制端口。

解析：端口是寄存器等软件可访问对象，不等同于板上引脚。

来源：教材第6章 · 正文 p258-259

F6-02 复习课件例题

程序查询方式在传输前由 CPU 读取接口的状态寄存器。

解析：外设准备好后再读写数据。

来源：教材第6章 · 正文 p262；期末复习总结 · 填空例题与答案

F6-03 真题改编

查询传输的接口硬件至少需要状态端口和数据端口。

解析：题目没有要求每个简单接口都必须有独立控制口。

来源：教材第6章 · 正文 p262；往届 A 卷 · 填空题

F6-04 教材巩固

外设始终可接受访问、不需先检查状态的方式叫无条件传送。

解析：课堂把实验二拨码与显示作为此方式的例子。

来源：教材第6章 · 正文 p259,261-262

F6-05 教材巩固

外设通过请求 CPU 暂停当前程序并执行服务程序来传输，属于程序中断方式。

解析：外设发中断请求，不等于外设变成总线主机。

来源：教材第6章 · 正文 p263-266

F6-06 教材巩固

中断前程序正在使用的状态标志及相关寄存器内容称为现场。

解析：中断结束后应恢复，使被中断程序能够正确继续。

来源：教材第6章 · 正文 p264

F6-07 教材巩固

中断返回时主程序继续执行的位置称为中断断点。

解析：与调试器设置的断点含义不同。

来源：教材第6章 · 正文 p264

F6-08 真题改编

DMA 传完一批数据后常用中断方式通知 CPU。

解析：批量数据搬运由 DMA 控制器承担。

来源：教材第6章 · 正文 p267-269；往届 A 卷 · 填空题

F6-09 实验二

数码管动态显示通过依次选择不同位，利用视觉暂留显示多位数字。

解析：必须持续刷新；写一次位选不会自动完成软件扫描。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-10 实验二

本实验二段选为高有效，位选为低有效。

解析：以 code_def.h 的定义为准，不套用其他板卡电平。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-11 实验二

实验二关闭四位数码管时，位选寄存器低四位应写 0x0F。

解析：低有效位选全部为 1 时均关闭。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-12 实验二

动态扫描改变段码前先关闭位选，主要用于减少显示重影。

解析：推荐“关位选→改段码→开目标位→保持”。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-13 实验二

读取 8 位开关后，若输入为二进制 10000000，十进制显示应为128。

解析：实验二要求将无符号二进制转换成十进制显示。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-14 实验三

加速器驱动通过写控制寄存器发出启动、清除等命令。

解析：命令位含义取决于实际接口协议。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-15 实验三

实验三 STATUS 中 BUSY、DONE、ERROR 依次位于 bit0、bit1、bit2。

解析：DONE 的掩码为 2，ERROR 掩码为 4，不可写成位序号。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-16 实验三

实验三 CTRL 的 ABORT 命令写入数值为2。

解析：bit1=1；开始前清旧状态，失败时中止，防止后续读到旧结果。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-17 实验三

实验三 SIN_RESULT 的字节偏移为 0x0C。

解析：基地址 0x40000000，SIN 位于 +0x0C，COS 位于 +0x10。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-18 实验三

实验三驱动轮询必须设置次数上限，以避免外设故障导致程序死循环。

解析：每次循环应重新读取 STATUS，超时退出并清理。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-19 实验三

实验三的 sin/cos 结果隐含三位小数，显示 0866 表示数值0.866。

解析：结果页 DOT 关闭，负号由 LED2 指示。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-20 实验三

实验三定点后处理使用64位中间值来完成乘 1000 等计算与舍入。

解析：Q30 值乘 1000 可能超过 32 位；舍入为零时应关闭负号。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

P01 / 纯汇编 / 5 空 / 真题改编

统计二进制 1 的个数

uint32_t count_ones(uint32_t n)：R0 传入任意 32 位无符号数，R0 返回其二进制 1 的个数。将往届固定 n=14 的代码改为可调用函数。

C调用示例（已给）

```
extern uint32_t count_ones(uint32_t n);
uint32_t bits = count_ones(0x80007001u); /* bits=5 */
```

逐空参考答案

(1) done (2) R2 (3) R1

(4) #1 (5) loop

为什么这样填

先掩码最低位再累加，逻辑右移后循环；用 BEQ 判断零，避免把最高位为 1 的无符号输入当负数提前结束。 迁移到C调用时应先看函数原型，不能固定认为输入必须是R7或结果必须留在R5；P21给出同一统计目标的掩码扫描写法。

验证案例

- n=0 → 0
- n=14 → 3
- n=0x80000000 → 1
- n=0xFFFFFFFF → 32

来源：往届 A 卷 · 编程 1；教材 §4.1 · 位运算与移位；用户补充参考图5 · 另一种位统计约定；用户补充参考图6 · 掩码扫描，可与P21对照

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P02 / 纯汇编 / 5 空 / 课件改编

寻找 16 位最高有效位

输入 R0 为 1-65535，返回最高位 1 的位序到 R0（最低位编号 0）。用 R1=15 从高位向低位扫描。

逐空参考答案

(1) #15 (2) =0x8000 (3) found

(4) #1 (5) scan

为什么这样填

掩码从 bit15 开始，未命中则掩码与位序同时右移/减一。输入严格大于零；若扩展到零，需要另设返回约定。

验证案例

- 1 → 0
- 12 → 3
- 0x8000 → 15

来源：期末复习总结 · 最高位例题

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P03 / 纯汇编 / 5 空 / 实验一改编

拆分半字并按指定地址存储

实验一的安全化变式：R0 为 32 位输入，高 16 位为 A，低 16 位为 B；R1 为有效且 4 字节对齐的目标 RAM 指针。计算 A+B×A，写入 *R1 并从 R0 返回（保留低 32 位）。

逐空参考答案

(1) #16 (2) R0 (3) R2

(4) [R1] (5) LR

为什么这样填

LSRS 提取高半字，UXTH 提取低半字；Thumb 乘法这里使用目的寄存器与末操作数相同的形式。原实验按学号构造的地址未必对齐或落入 RAM，练习改为显式传入合法指针。

验证案例

- 0x00030004 → 15 且 *R1=15
- 0x00000009 → 0
- 0xFFFF0001 → 131070

来源：实验一 · 任务1；教材 §4.1 · 算术和访存

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P04 / 纯汇编 / 5 空 / 教材变式

累加数组中的无符号字

uint32_t sum_words(const uint32_t *p,uint32_t n)。R0=p，R1=n；n=0 返回 0，否则累计 n 个字，按模 2^32 返回。指针有效且字对齐。

逐空参考答案

(1) [R0] (2) R3 (3) #4

(4) BNE (5) R2

为什么这样填

每个元素为 4 字节。SUBS 与 BNE 之间不放更新标志的其他指令。

验证案例

• n=0 → 0

• [1,2,3] → 6

• [0xFFFFFFFF,1] → 0

来源：教材 §4.2.3 · 循环结构；复习记录 · 37:45-44:22 程序填空要求

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P05 / 纯汇编 / 5 空 / 教材变式

减法式最大公约数

uint32_t gcd_sub(uint32_t a,uint32_t b)。R0=a、R1=b，均为 1-65535；用反复相减求 GCD，R0 返回结果。比较均按无符号处理。

逐空参考答案

(1) done (2) BCC / BLO (3) R1

(4) R0 (5) loop

为什么这样填

a>b 时 a-=b，a<b 时 b-=a，直到相等；零输入未在合同中允许，不能拿此算法直接处理 a=0 或 b=0。

验证案例

- 48,18 → 6
- 7,7 → 7
- 15,4 → 1

来源：教材 §2.5.1 · GCD 算法；教材 §6.3.2 · GCD 加速器

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P06 / 混合编程 / 10 空 / 复习课件改编

C 调汇编：斐波那契

C 原型 uint32_t fibonacci(uint32_t a,uint32_t b,uint32_t n)。a=b=1，2≤n≤46；R0/R1/R2 传参，返回第 n 项。补全 ARMASM 结构和递推循环。

C调用示例（已给）

```
extern uint32_t fibonacci(uint32_t a,uint32_t b,uint32_t n);
uint32_t f = fibonacci(1u,1u,10u); /* f=55 */
```

逐空参考答案

(1) AREA (2) EXPORT / GLOBAL (3) #2

(4) R1 (5) R1 (6) R3

(7) #1 (8) BNE (9) R1

(10) LR

为什么这样填

R0/R1 保存连续两项，R3 为新项，循环次数 n-2。课件举 F(50)，但其数学结果 12586269025 超过 32 位；本题限制 n≤46，消除有符号/无符号溢出歧义。

验证案例

- n=2 → 1
- n=10 → 55
- n=46 → 1836311903

来源：期末复习总结 · Fibonacci 例题；重点复习记录 · 01:14:50-01:21:46

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P07 / 混合编程 / 10 空 / 课件校订改编

汇编调用 C：两日期间隔

C 函数 int date_index(int year,int month,int day) 已实现，year=年份后两位；返回可相减的日期序号。汇编函数 gap_days(void) 计算 2023-06-15 减去 2000-01-01 的天数。C 函数按调用约定可改写 R0-R3。

逐空参考答案

(1) AREA (2) IMPORT / EXTERN (3) LR

(4) #23 (5) #6 (6) #15

(7) BL (8) R0 (9) R4

(10) PC

为什么这样填

两次调用之间用被调用者保存寄存器 R4 保留首个结果。PUSH 两个寄存器共 8 字节，保持跨 C 调用的栈对齐。课件答案的“#R0”是笔误，应为 R0。

验证案例

• gap_days() → 8566 天

来源：期末复习总结 · 日期混合编程例题；教材 §4.2.4 · 混合编程

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P08 / 混合编程 / 10 空 / 真题校订改编

递归阶乘与启动结构

补全原卷风格的启动框架与 uint32_t factorial(uint32_t n)。输入 0≤n≤12；复位后计算 5!。Stack_Size=0x400，栈段 ALIGN=3。

逐空参考答案

(1) SPACE (2) __initial_sp (3) BL

(4) LR (5) R0 (6) CMP

(7) R4 (8) factorial (9) MULS

(10) PC

为什么这样填

保存 R4 和 LR；R4 在递归期间保存本层 n；基例 0! 与 1! 均为 1。返回值经 R0，POP PC 完成本层返回。

验证案例

• 0 → 1

• 5 → 120

• 12 → 479001600

来源：往届 A 卷 · 编程2；教材 §5.3.2 · 向量表与启动

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P09 / 混合编程 / 10 空 / 实验一改编

实验一：汇编调用累加 C 函数并求商

已给 C 函数 uint32_t add(uint32_t a)，返回 1+…+a。汇编函数 mean_sum(uint32_t a) 在 1≤a≤99 时计算 add(a)/a 的整数商。Cortex-M0 不使用 UDIV，改用重复减法。

逐空参考答案

(1) IMPORT / EXTERN (2) PUSH (3) R0

(4) BL (5) #0 (6) BCC / BLO

(7) R4 (8) #1 (9) R1

(10) PC

为什么这样填

R4 保存除数以跨越 C 调用；余数放 R0，商放 R1。返回 floor((a+1)/2)，偶数 a 的结果要向下取整。

验证案例

• a=1 → 1

• a=12：B=78，返回6

• a=23：B=276，返回12

来源：实验一 · 任务2；实验一 · 已给 add C 函数

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P10 / 混合编程 / 10 空 / 课件校订改编

反复调用 C：6174 黑洞步数

C 函数 uint32_t digitSort(uint32_t n) 已给：保留四位十进制（不足补零），降序数减升序数。输入为 0-9999 且至少两位不同。kap_steps 返回首次到达 6174 所需步数；输入已为 6174 时返回 0。

C调用示例（已给）

```
extern uint32_t kap_steps(uint32_t n);
uint32_t steps = kap_steps(3524u); /* steps=3 */
```

逐空参考答案

(1) IMPORT / EXTERN (2) LR (3) R0

(4) #0 (5) =6174 (6) R4

(7) BL (8) #1 (9) loop

(10) PC

为什么这样填

计数器放 R5，不能假设 R1 在 C 调用后不变；每做一次变换即加一，初值已等于目标则不调用。保存四个寄存器共16字节。课件原片段的 R1 计数、少算末步及栈对齐问题在此修正。 已给digitSort的内部排序不是本题填空目标；真正要跟踪的是参数送入R0、BL后结果取回、跨调用状态保存和步数定义。

验证案例

- 3524 → 3
- 6174 → 0
- 1000 → 5

来源：第5讲 · PDF p53-55；用户补充参考图7 · 给定C函数digitSort；用户补充参考图8 · 混合调用框架

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P11 / 混合编程 / 10 空 / 课件校订改编

C 调汇编：安全容量下拼接字符串

void append_text(const char *src,char *dst)。R0=src，R1=dst；两区不重叠，dst 容量足够且均有零结尾。将 src 拼到 dst 末尾并保留结尾零。用显式指针递增满足 Cortex-M0 指令形式。

C调用示例（已给）

```
extern void append_text(const char *src,char *dst);
char dst[32] = "Hello";
append_text("World!",dst); /* dst="HelloWorld!" */
```

逐空参考答案

(1) EXPORT / GLOBAL (2) LDRB (3) #0

(4) #1 (5) [R0] (6) STRB

(7) #0 (8) BNE (9) BX

(10) LR

为什么这样填

先找到 dst 的零字节，再逐字节复制 src，零也要复制。课件示例 char b[]="Hello" 没有拼接余量；练习调用时应用 char b[32]="Hello"。避免课件中 Thumb-2 风格的访存后索引写法。 这里append_text的(src,dst)顺序沿用课程自定义函数，不能套用标准库strcat的(dst,src)顺序；也不使用strcat名称，以免与标准库原型冲突。

验证案例

- src="World!", dst="Hello" → "HelloWorld!"
- 空 src：dst不变
- 空 dst：复制整个src

来源：第5讲 · PDF p56-58；用户补充参考图9 · C调用者与缓冲容量；用户补充参考图10 · 汇编字符串拼接框架

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P12 / 软硬件综合 / 10 空 / 教材与真题变式

GPIO 单次流水灯扫描

接口由题面定义：GPIO_BASE=0x40010000，+0x00 为输出数据，+0x04 为方向（bit=1 输出）；8 个 LED 高有效。C 函数 delay_c(uint32_t ticks) 已给。led_scan 完成 01→02→…→80 后熄灭返回；不要求精准延时。

逐空参考答案

(1) IMPORT / EXTERN (2) LR (3) =0x40010000

(4) #0xFF / #255 (5) #8 (6) R5

(7) BL (8) #1 (9) BNE

(10) PC

为什么这样填

R4-R6 在 C 调用后保持不变，保存基地址、位图与循环计数。方向定义和电平只用于本题，不套用实验二。

验证案例

- 数据写序列：1,2,4,8,16,32,64,128,0
- delay_c 调用8次

来源：教材 §5.4.2 · GPIO 流水灯；往届 A 卷 · 编程3 GPIO背景

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P13 / 软硬件综合 / 10 空 / 真题校订改编

PWM 周期与 C 调用保护

void pwm_once(uint32_t led,uint32_t duty)。输入 R0=LED位图，R1=0-100 占空比；高有效 GPIO 数据口为 0x40010000。delay_pwm_c(ticks) 已给，允许 ticks=0。先亮 duty，再灭 100-duty，完成一个概念 PWM 周期。

逐空参考答案

(1) IMPORT / EXTERN (2) LR (3) R0

(4) R1 (5) =0x40010000 (6) [R6]

(7) BL (8) #100 (9) R5

(10) PC

为什么这样填

把跨 C 调用要保留的变量放在 R4-R6，入栈16字节保持8字节对齐。原卷 pwm_cycle 的 PUSH {LR} 后跨 C 调用未保持8字节对齐，本题修正。空循环与调用有开销，本题不承诺物理 PWM 精度。

验证案例

- duty=25：参数25、75，输出led后输出0
- duty=0：参数0、100
- duty=100：参数100、0

来源：往届 A 卷 · 编程3 PWM；教材 §4.2.4 · 混合编程

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P14 / 软硬件综合 / 10 空 / 实验二改编

实验二：开关值分解为三位十进制

void switch_digits(uint8_t *out)。R0 指向至少3字节RAM，按百、十、个位写出数值0-9。SWITCH_DATA=0x40000000，仅低8位有效。不使用除法指令，采用重复减法。

逐空参考答案

(1) =0x40000000 (2) #255 / #0xFF (3) #100

(4) [R0] (5) #10 (6) BCC / BLO

(7) #10 (8) [R0,#1] (9) [R0,#2]

(10) LR

为什么这样填

先屏蔽高24位，然后减100求百位，再减10求十位，剩余为个位。这里写出的是数字值，尚需段码表译码后才能送显示口。

验证案例

- 0 → [0,0,0]
- 128 → [1,2,8]
- 255 → [2,5,5]

来源：实验二要求；实验二 · 十进制转换与显示流程

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P15 / 软硬件综合 / 10 空 / 实验二改编

实验二：按段选与位选完成一帧扫描

void scan_frame(const uint8_t *codes)。R0 指向百、十、个位的段码（已译码，表按输入定义提供）；SEG_LED=0x40000010，SEG_SEL=0x40000014，段高有效、位低有效。位选依次0x0B、0x0D、0x0E；delay_c(20)已给。本题统一显示三位，不消前导零。

逐空参考答案

(1) =0x40000010 (2) #15 / #0x0F (3) #4

(4) LDRB (5) [R5] (6) BL

(7) #1 (8) BNE (9) #8

(10) PC

为什么这样填

栈上计数不能放易被 C 函数破坏的 R0-R3 中；STR 不改条件标志，SUBS 后的 BNE 仍读取其 Z。每次先关闭所有位，更新段码后再打开目标位。需由主循环反复调用实现持续显示。

验证案例

- 每帧3次delay调用
- 段码按百、十、个顺序写入
- 最后SEG_SEL=0x0F

来源：实验二 · 地址和位选定义；实验二 · 先关后开扫描

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P16 / 软硬件综合 / 10 空 / 实验三改编

实验三：带超时与清理的加速器驱动

int trig_hw_run(int32_t angle,int32_t *s,int32_t *c)。R0/R1/R2传参，输出指针合法且字对齐。基址0x40000000；ANGLE +0、CTRL +4、STATUS +8、SIN +0x0C、COS +0x10；CTRL bit0 START、bit1 ABORT；STATUS bit0 BUSY、bit1 DONE、bit2 ERROR。成功返回0，超时-1，硬件错误-2；仅成功写输出，失败先ABORT。

逐空参考答案

(1) =0x40000000 (2) #2 (3) #0

(4) #8 (5) #4 (6) #2

(7) BNE (8) #0x0C / #12 (9) #0x10 / #16

(10) PC

为什么这样填

每次都读新 STATUS，先检查 ERROR 再检查 DONE；有限轮询避免卡死。启动前 ABORT 清旧完成标志，失败路径不覆盖调用者输出。角度是预处理后的Q30值，范围0到π/2；符号恢复及64位舍入由已给C函数负责。

验证案例

• DONE：R0=0，两个输出更新

• ERROR：R0=-2，输出不变，CTRL最后写2

• 持续BUSY：4096次后-1并ABORT

来源：实验三 · 任务2与任务3；实验三 · AHB加速器寄存器；实验三 · 汇编驱动接口

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P17 / 软硬件综合 / 10 空 / 课堂要求变式

读 Verilog 译码后编写握手驱动

读给定硬件片段，补全一次查询式交易。端口不等待，总线地址相位已锁存。reg_id=0为写入数据，1为控制（写1启动），2为状态（bit0完成），3为结果。接口调用：R0输入，R1输出指针；最多100次状态读取，返回0成功、-1超时。代码须依据译码算出窗口基址与寄存器字节偏移。

逐空参考答案

(1) =0x40000300 (2) #0 (3) #1

(4) #4 (5) #8 (6) BCS / BHS

(7) #12 / #0x0C (8) [R1] (9) #0

(10) LR

为什么这样填

高24位译码对应256字节窗口；HADDR[3:2] 为字索引，地址需乘4。LSRS #1 把原 bit0 移入 C，BCS 判断完成。题设控制器在新启动时清 DONE；不能自动套用实验三的 ABORT 协议。

验证案例

- 译码窗口0x40000300-0x400003FF
- 首次状态bit0=1：读+12写*R1，返回0
- 100次未完成：-1，*R1不变

来源：重点复习记录 · 39:45-44:22 读硬件写汇编；实验三 · 地址相位/数据相位实例；教材 §5.2 · 全译码与映射

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P18 / 软硬件综合 / 10 空 / 课堂方案变式

中断方案：读取结果、清源与设置标志

课堂拓展的中断方案，和实验三轮询方案分开。假定IRQ=1已正确使能并向量绑定：结果端口0x4000040C；清中断端口0x40000410写1后外设撤销IRQ。result与done是C端已定义的volatile uint32_t全局量。处理函数不调用其他函数，只用硬件自动堆栈的R0-R3；读取结果后清中断源，再置done=1并异常返回。

逐空参考答案

(1) IMPORT / EXTERN (2) EXPORT / GLOBAL (3) =0x4000040C

(4) LDR (5) =result (6) [R2]

(7) #1 (8) =done (9) BX

(10) LR

为什么这样填

硬件保存基础现场；LR 此时为 EXC_RETURN 特殊值，BX LR触发异常返回。这里不额外PUSH且不BL，因此不会破坏该值；DSB确保清源访问完成后返回。软件完成标志不能代替清除外设中断源。地址与写1清除语义是本题另给的假设，不属于实验三现有接口。

验证案例

- 返回前：result为硬件结果、done=1
- 对0x40000410写1撤销IRQ
- BX LR使用EXC_RETURN恢复被中断程序

来源：重点复习记录 · 26:16-30:36 中断与加速器；教材 §6.1.2 与 §6.3.2 · 中断

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P19 / 混合编程 / 5 空 / 补充参考变式

C 调汇编：任意长度数组的升序检查

C 原型 uint32_t is_sorted(const int32_t *a,uint32_t n)。检查非严格升序，返回1或0；比较按有符号32位数处理。R0=a，R1=n。n为0或1时返回1；n=0不解引用a，n>0时a指向至少n个有效且字对齐的元素。n=5对应参考题，函数也必须处理其他长度。

C调用示例（已给）

```
extern uint32_t is_sorted(const int32_t *a,uint32_t n);
int32_t a[] = {-3,-3,0,8,8};
uint32_t ok = is_sorted(a,5u); /* ok=1 */
```

逐空参考答案

(1) BGT (2) R3 (3) R1

(4) BNE (5) #1

为什么这样填

指针参数通过R0传入的是首地址，n通过R1传入；每次LDR读取4字节，n个元素只需n-1次相邻比较。CMP R2,R3后BGT在“前项有符号大于后项”时失败，相等允许通过。R2须更新为当前项才能检查下一对。返回值写R0，BX LR回到C调用者；全程只借用R0-R3。

验证案例

- []，n=0 → 1，允许a=NULL
- [-3,-3,0,8,8] → 1
- [1,3,2,4,5] → 0
- [INT32_MIN,INT32_MAX] → 1

来源：用户补充参考图1 · 长度5的升序题意；用户补充参考图2 · 相邻元素比较框架；教材 §4.2.4 · C/汇编调用

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P20 / 混合编程 / 10 空 / 补充参考变式

C 调汇编：两个数组逐项取小取大

C 原型 void split_minmax(uint32_t *a,uint32_t *b,uint32_t n)。将每对元素中较小值写回a[i]、较大值写回b[i]，按无符号32位数比较。R0=a、R1=b、R2=n；0≤n≤65535，两数组有效、字对齐且不重叠，n=0不访存。数组可写；参数指针的数值与数组元素内容不可混淆。

C调用示例（已给）

```
extern void split_minmax(uint32_t *a,uint32_t *b,uint32_t n);
uint32_t a[] = {12,35,0}, b[] = {24,25,0};
split_minmax(a,b,3u); /* a={12,25,0}, b={24,35,0} */
```

逐空参考答案

(1) EXPORT / GLOBAL (2) LR (3) #2

(4) [R0,R3] (5) CMP (6) BLS

(7) [R0,R3] (8) #4 (9) BCC / BLO

(10) PC

为什么这样填

R3是字节偏移，R6=4n；两次LDR先取出两个旧值，再按a[i]>b[i]交换，避免提前写回丢失数据。BLS允许相等时跳过交换。借用了R4-R6，必须在返回C前恢复。此函数是void，结果通过两个数组的内存内容体现，不必把结果塞进R0。参考截图的DB文字与DCD代码不一致，本题明确使用uint32_t和4字节步长。

验证案例

- a=[12,35,0]，b=[24,25,0] → a=[12,25,0]，b=[24,35,0]
- a=[0xFFFFFFFF]，b=[1] → a=[1]，b=[0xFFFFFFFF]
- n=0：两数组均不访问
- 相等元素保持不变

来源：用户补充参考图3 · 两数组对应位置比较；用户补充参考图4 · 字数组与地址偏移框架；教材 §4.2.4 · 调用与寄存器保存

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P21 / 混合编程 / 5 空 / 补充参考变式

C 调汇编：用移动掩码统计32位1

C 原型 uint32_t count_mask(uint32_t value)。R0传入任意32位值，以1、2、4、…、0x80000000逐位测试，R0返回1的个数。输入数据放R2保持不变。仅填汇编；不能把参考题的“R7输入、R5输出”直接当成C调用规则。

C调用示例（已给）

```
extern uint32_t count_mask(uint32_t value);
uint32_t bits = count_mask(0x80007001u); /* bits=5 */
```

逐空参考答案

(1) BEQ (2) #1 (3) #1

(4) BNE (5) LR

为什么这样填

R1为扫描掩码，R2为不变输入，R0为累计结果。ANDS等于零则不计数；LSLS掩码后更新Z，BNE据此继续。最高位测试完成后掩码再左移会变为零，恰好终止32次扫描，不漏bit31。这里不能在LSLS与BNE之间插入改变标志的指令。与P01右移数据的实现有相同C接口思路但不同循环不变量。

验证案例

- 0x80007001 → 5
- 0 → 0
- 0x80000000 → 1
- 0xFFFFFFFF → 32

来源：用户补充参考图5 · R7/R5位计数题意；用户补充参考图6 · 移动掩码思路；往届 A 卷 · 位统计背景

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P22 / 混合编程 / 10 空 / 补充参考变式

迁移题：C 调汇编统计字符出现次数

C 原型 uint32_t count_char(const char *s,uint32_t ch)。s为有效、零结尾的字符串；ch在0-255。仅统计终止零之前与ch相等的字节，不改写字符串；ch=0时返回0。R0=s，R1=ch；R0返回次数。例：count_char("banana",'a')返回3。

C调用示例（已给）

```
extern uint32_t count_char(const char *s,uint32_t ch);
uint32_t n = count_char("banana",(uint32_t)'a'); /* n=3 */
```

逐空参考答案

(1) EXPORT / GLOBAL (2) #0 (3) LDRB

(4) #0 (5) done (6) BNE

(7) #1 (8) #1 (9) R2

(10) LR

为什么这样填

与字符串拼接同样接收指针、按字节遍历、识别结尾零，但本题不写内存。R0和R1分别承接C的两个实参；每个字符使用LDRB读取，指针加1。先判断终止零再判断目标字符，确保不把结尾零统计进去。算法变化后，C/汇编接口方法仍然相同。

验证案例

- "banana"，ch='a' → 3
- 空字符串 → 0
- "abc"，ch=0 → 0
- "aaaa"，ch='z' → 0

来源：用户补充参考图9 · C调用汇编的字符串背景；用户补充参考图10 · 字节访存与结尾零；C调用汇编迁移题

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

P23 / 混合编程 / 10 空 / 补充参考变式

迁移题：C 调汇编完成带容量的字符串复制

C 原型 int32_t copy_z(const char *src,char *dst,uint32_t cap)。R0=src，R1=dst，R2=cap，0≤cap≤65535。src有效且零结尾，cap>0时dst至少有cap字节且不与src重叠。完整复制时返回字符数（不含结尾零）；空间不足返回-1，cap>0时仍保证dst零结尾。cap=0不写dst并返回-1。只填汇编，不填写C。

C调用示例（已给）

```
extern int32_t copy_z(const char *src,char *dst,uint32_t cap);
char dst[4];
int32_t n = copy_z("abc",dst,sizeof dst); /* n=3 */
```

逐空参考答案

(1) EXPORT / GLOBAL (2) #0 (3) #0

(4) LDRB (5) BEQ (6) #1

(7) STRB (8) #1 (9) R4

(10) PC

为什么这样填

cap必须包含结尾零所占的一字节。R4统计实际复制的非零字符，cap=1且源仍有字符时，仅能写零并返回-1。先识别源串终止零，再检查cap=1，可使“恰好放得下”的情况成功。R0负责返回状态/长度；dst内存承接字符串，二者不是同一种输出。R4为被调用者保存寄存器，已配对保护。

验证案例

- src="abc"，cap=4 → dst="abc"，返回3
- src="abc"，cap=3 → dst="ab"，返回-1
- src=""，cap=1 → 空串，返回0
- cap=0 → 不写dst，返回-1

来源：用户补充参考图9 · 原题目标数组容量不足；用户补充参考图10 · 从字节复制迁移到带容量接口

指令与边界提醒

参考填法必须与给定上下文配合。等价改写若改变寄存器、标志或调用约定，需检查所有路径；不能只比较最后一次计算结果。各题外设地址和协议独立，以本题题面为准。

校订说明与使用边界

资料的优先顺序

本学期讲义用于概念表述，重点复习记录与复习总结用于确定练习形式，实验要求与当前接口代码用于确定实验变式的寄存器、状态与电平。往届原始带答案 DOC 优先于现有 Markdown 的照片重建。

Fibonacci 的范围

复习课件的 F(50)=12586269025，无法用32位整数完整表示。本题 P06 限 n≤46，结果在32位有符号数范围内，避免把截断值当数学答案。

C 调用与栈

R0-R3 不应当被假定跨 C 调用保持不变。涉及调用的参考答案将持久变量放在被调用者保存寄存器或栈上，并在调用边界保持8字节栈对齐。P13 修正了原卷 PWM 片段只压 LR 的对齐问题。

课件程序校订

P07 将“#R0”改成寄存器操作数 R0；P10 修正计数器跨C调用、末步计数和栈对齐；P11 明确目标缓冲容量，使用 Cortex-M0 可用的显式指针递增。

两种加速器方案

P16 采用实验三当前轮询接口，包含 ABORT、DONE 和 ERROR。P18 是课堂中断方案的独立变式，写1清源地址由题面给定，不能当作原实验寄存器。

关于填空匹配

网页按列出的参考答案和同义写法核对。程序等价写法可能多于题库列举范围，因此可查看解析后自评；这不代表已执行或硬件验证。

关于覆盖

本题库覆盖六章的基础概念与三类程序练习。第二章仅保留读硬件所需基础；不编写 Verilog，不含简答。老师的复习提点不作为保证命中的考点清单。